

TASK 1

1) There are more ways than 2 to determine the mounted disk:

- a) `lsblk` - it shows all disks; however, we can look at and determine which disk is mounted or not here:

```
[root@rhel-9 ~ #] lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 12.7G  0  rom  /mnt
nvme0n1              259:0    0   10G  0  disk
├─nvme0n1p1          259:1    0   600M  0  part /boot/efi
├─nvme0n1p2          259:2    0    1G  0  part /boot
├─nvme0n1p3          259:3    0   8.4G  0  part
└─┬─rhel-root        253:0    0   8.4G  0  lvm  /
   └─nvme0n2          259:4    0    6G  0  disk
      └─nvme0n2p1     259:5    0    6G  0  part
```

In that example, `sr0` has mounted to `/mnt` directory, we can see it easily. However, there is no mountpoint for `nvme0n2p1` partition of disk.

- b) `findmnt` - It shows mounted filesystems in the system with a tree view. It shows also options which we can define in `/etc/fstab`.

```
[root@rhel-9 ~ #] findmnt
TARGET                SOURCE                FSTYPE  OPTIONS
├─/proc                /dev/mapper/rhel-root xfs     rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota
├─┬─/proc/sys/fs/binfmt_misc
│  └─proc               systemd-1            proc     rw,nosuid,nodev,noexec,relatime
├─┬─/sys
│  └─/sys/kernel/security sysfs                sysfs    rw,relatime,fd=29,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=25716
├─┬─/sys/fs/cgroup      cgroup2              cgroup2  rw,nosuid,nodev,noexec,relatime,seclabel
├─┬─/sys/fs/pstore      pstore               pstore   rw,nosuid,nodev,noexec,relatime,seclabel
```

- c) `df -h` - It also shows mounted file systems. As extra, it gives info about disk space, how much is used or available:

```
[root@rhel-9 ~ #] df -h
Filesystem            Size  Used Avail Use% Mounted on
devtmpfs              4.0M   0    4.0M   0% /dev
tmpfs                 838M   0    838M   0% /dev/shm
tmpfs                 335M   6.4M  329M   2% /run
efivarfs              256K   57K   195K  23% /sys/firmware/efi/efivars
/dev/mapper/rhel-root  8.4G   2.4G   6.1G  29% /
/dev/nvme0n1p2        960M  244M  717M  26% /boot
/dev/nvme0n1p1        599M   7.4M  592M   2% /boot/efi
tmpfs                 168M   4.0K  168M   1% /run/user/0
/dev/sr0              13G   13G    0 100% /mnt
```

- d) `mount` (without any option) - I have found another command which also can help us. It is not quite readable but again possible to consider as another command to see mounted file systems:

```
[root@rhel-9 ~ #] mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime,seclabel)
devtmpfs on /dev type devtmpfs (rw,nosuid,seclabel,size=4096k,nr_inodes=208485,mode=755,inode64)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,seclabel,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,seclabel,gid=5,mode=620,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,seclabel,size=342852k,nr_inodes=819200,mode=755,inode64)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,seclabel,nsdelegate,memory_recursiveprot)
pstore on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime,seclabel)
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)
```

2) I have explained them above. Their outputs:

```
[root@rhel-9 ~ #] lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                                11:0    1 12.7G  0  rom  /mnt
nvme0n1                            259:0    0   10G  0  disk
├─nvme0n1p1                        259:1    0  600M  0  part  /boot/efi
├─nvme0n1p2                        259:2    0    1G  0  part  /boot
├─nvme0n1p3                        259:3    0   8.4G  0  part
└─rhel-root                        253:0    0   8.4G  0  lvm  /
nvme0n2                            259:4    0    6G  0  disk
└─nvme0n2p1                        259:5    0    6G  0  part
nvme0n3                            259:6    0    5G  0  disk
└─nvme0n3p1                        259:7    0    5G  0  part
    └─vg02-lvswap                  253:1    0    2G  0  lvm  [SWAP]
```

```
[root@rhel-9 ~ #] findmnt
TARGET                                SOURCE                                FSTYPE  OPTIONS
/                                     /dev/mapper/rhel-root                xfs     rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota
├─/proc                             proc                                 proc     rw,nosuid,nodev,noexec,relatime
├─/proc/sys/fs/binfmt_misc          systemd-1                            autofs   rw,relatime,fd=29,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=25716
├─/sys                              sysfs                                sysfs    rw,nosuid,nodev,noexec,relatime,seclabel
├─/sys/kernel/security              securityfs                            securityfs rw,nosuid,nodev,noexec,relatime
└─/sys/fs/cgroup                    cgroup2                              cgroup2  rw,nosuid,nodev,noexec,relatime,seclabel,nsdelegate,memory_recursiveprot
```

`lsblk` shows all disks and which ones are mounted. `findmnt` shows only mounted disks with an extra info like their mount options as a tree view.

3) To add a new disk, we need to go Settings in VMWare Workstation Pro and add new hard disk. We define its volume (5 and 4 GB). I add it as HDD, not SDD. To see them, we need to run `lsblk` command:

```
[root@rhel-9 ~ #] lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                 8:0     0    5G  0  disk
sdb                                 8:16    0    4G  0  disk
sr0                                11:0     1 12.7G  0  rom  /mnt
nvme0n1                            259:0    0   10G  0  disk
├─nvme0n1p1                        259:1    0  600M  0  part  /boot/efi
├─nvme0n1p2                        259:2    0    1G  0  part  /boot
├─nvme0n1p3                        259:3    0   8.4G  0  part
└─rhel-root                        253:0    0   8.4G  0  lvm  /
```

`sda` (5G) and `sdb` (4G) are new added disks.

4) First, we need to create a partition. Let's use `fdisk` (MBR) to create a partition:

```
[root@rhel-9 ~ #] fdisk /dev/sda

Welcome to fdisk (util-linux 2.37.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0x8f9d198e.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (1-4, default 1):
First sector (2048-10485759, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-10485759, default 10485759):

Created a new partition 1 of type 'Linux' and of size 5 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

As you can see, I have created one partition and give it full disk space. We can check the newly created partition by just running `lsblk` command.

```
[root@rhel-9 ~ #] lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   5G  0 disk
└─sda1       8:1    0   5G  0 part
```

Then we need to create PV for our partition, and then VG. The command to create VG is `vgcreate` and this also creates PV for us, so we do not need to create PV manually:

```
[root@rhel-9 ~ #] vgcreate vg_test /dev/sda1
Physical volume "/dev/sda1" successfully created.
Volume group "vg_test" successfully created
[root@rhel-9 ~ #] vgs
VG      #PV #LV #SN Attr   VSize  VFree
rhel    1   1   0 wz--n-  8.41g    0
vg_test 1   0   0 wz--n- <5.00g <5.00g
```

As you can see above, by running `vgs` (`vgdisplay` for more info) command, we can see all `VG`'s available in the system and `vg_test` is here.

If we run `pvs` (`pvdiskdisplay` for more info), we will also see PVs available in the system:

```
[root@rhel-9 ~ #] pvs
PV          VG      Fmt  Attr PSize  PFree
/dev/nvme0n1p3 rhel   lvm2 a--  8.41g    0
/dev/sda1    vg_test lvm2 a--  <5.00g <5.00g
```

5) We have a command `lvcreate` to create logical volumes:

```
[root@rhel-9 ~ #] lvcreate -n lvtest1 -L 2G vg_test
Logical volume "lvtest1" created.
[root@rhel-9 ~ #] lvcreate -n lvtest2 -l 100%FREE vg_test
Logical volume "lvtest2" created.
[root@rhel-9 ~ #] lvs
LV      VG      Attr      LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
root    rhel    -wi-ao---- 8.41g
lvtest1 vg_test -wi-a----- 2.00g
lvtest2 vg_test -wi-a----- <3.00g
```

I have used `-L` to create the first logical volume. With `-L` we define exact volumes like `2G`, `100G` or `500M`, etc.

However, I have decided not to write `-L 3G` for the second logical volume, because it would not work. Why? Because LVs always take more space than we specify for them to store metadata.

JUST EXAMPLE: IF WE SPECIFY 6 GB FOR LV, IT TAKES 6,01 GB TO STORE METADATA (ITS UUID, VG IT BELONGS TO AND MORE)

In that case, we either need to write something like `-L 2.99G` or there is a better way which is using `-l 100%FREE`. `-l` is used for that kind of approximate cases. `-l 100%FREE` gives all free space available in the VG to our new LV.

If we use `lsblk` now, we will see our logical volumes:

```
[root@rhel-9 ~ #] lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                 8:0      0   5G  0 disk
└─sda1                             8:1      0   5G  0 part
   └─vg_test-lvtest1               253:1    0    2G  0 lvm
      └─vg_test-lvtest2           253:2    0    3G  0 lvm
```

6) Our commands to create ext4 and xfs filesystems are `mkfs.ext4` and `mkfs.xfs`.

```
[root@rhel-9 ~ #] mkfs.ext4 /dev/vg_test/lvtest1
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: f17f9a24-0ec9-437f-ab98-0f87c731f1a5
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

```
[root@rhel-9 ~ #] mkfs.xfs /dev/vg_test/lvtest2
meta-data=/dev/vg_test/lvtest2  isize=512    agcount=4, agsize=196352 blks
       =                       sectsz=512   attr=2, projid32bit=1
       =                       crc=1        finobt=1, sparse=1, rmapbt=0
       =                       reflink=1    bigtime=1 inobtcount=1 nnext64=0
data      =                       bsize=4096  blocks=785408, imaxpct=25
       =                       sunit=0      swidth=0 blks
naming    =version 2              bsize=4096  ascii-ci=0, ftype=1
log        =internal log          bsize=4096  blocks=16384, version=2
       =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime  =none                  extsz=4096  blocks=0, rtextents=0
```

- 7) I have used `mkdir` command to create directories and `touch` for files:

```
[root@rhel-9 ~ #] mkdir /dir1 /dir2
[root@rhel-9 ~ #] touch /dir1/ravan /dir2/ravan
[root@rhel-9 ~ #] ll /dir1
total 0
-rw-r--r--. 1 root root 0 Apr  1 20:45 ravan
[root@rhel-9 ~ #] ll /dir2
total 0
-rw-r--r--. 1 root root 0 Apr  1 20:45 ravan
```

- 8) There are two mount cases: temporary and permanent. To mount a disk temporarily, we just need to run `mount /dev/<DISK_NAME> /<MOUNTPOINT>`. However, if we need to mount it permanently, we need to add it to `/etc/fstab` file. Let's mount our disks permanently to `/dir1` and `/dir2` directories.

First, we need to open `/etc/fstab` and add these lines to end of the file:

```
UUID=f17f9a24-0ec9-437f-ab98-0f87c731f1a5 /dir1 ext4 defaults 0 0
UUID=b4fc3155-4074-4981-8586-2024a38b269f /dir2 xfs defaults 0 0
```

You may ask what the first UUIDs I wrote were? They are just UUIDs for logical volumes, we can use either logical volume paths or their UUIDs here. That is preferable to use UUID. I have used `blkid` command to find their UUIDs:

```
[root@rhel-9 ~ #] blkid | grep "vg_test"
/dev/mapper/vg_test-lvtest1: UUID="f17f9a24-0ec9-437f-ab98-0f87c731f1a5" TYPE="ext4"
/dev/mapper/vg_test-lvtest2: UUID="b4fc3155-4074-4981-8586-2024a38b269f" TYPE="xfs"
```

The second column specifies the directory where we mount our disk to. The third column is for filesystem type. The fourth column is for mount options; we prefer to write `defaults`. The fifth column determines whether we can take backup of the data on the disk or not. The last column represents the filesystem check at the boot.

After writing them to `/etc/fstab` files, we can restart the system, and the changes will have been applied. Or we have another method, we can just run this command to make changes applied:

```
[root@rhel-9 ~ #] systemctl daemon-reload && mount -a
```

- 9) In the folder where I have mounted a disk with `ext4`, a new directory is created inside it:

```
[root@rhel-9 ~ #] ll /dir1
total 16
drwx----- 2 root root 16384 Apr  1 21:19 lost+found
```

However, the second directory where I have mounted a disk with `xfs`, there is nothing:

```
[root@rhel-9 ~ #] ll /dir2
total 0
```

However, when we unmount and remove the lines from `/etc/fstab` and reboot the system, we will see the old contents (ravan file) in the directories, nothing like `lost+found` anymore.

Let's first determine what is `lost+found` directory. That is not about our case. It is always created when we mount a disk if our filesystem is `ext4`. Regardless of anything, it is just a part of the ext4 filesystem initialization itself. It is created by the fsck utility.

Now the main problem: Where were our files gone? Are they deleted?

Answer: In that kind of case the files under the directory are not deleted, they become hidden. The previous data (my ravan file) is stored on the **root**

filesystem - the same disk that holds our entire OS. When we unmount the disks, we will see `ravan` file in both directories again.

10) We use `touch` command to create files:

```
[root@rhel-9 ~ #] touch /dir1/muradzada && touch /dir2/muradzada
[root@rhel-9 ~ #] ll /dir1
total 16
drwx----- 2 root root 16384 Apr  1 21:19 lost+found
-rw-r--r-- 1 root root    0 Apr  1 21:53 muradzada
[root@rhel-9 ~ #] ll /dir2
total 0
-rw-r--r-- 1 root root 0 Apr  1 21:53 muradzada
```

11) I have `fdisk` command to divide a disk to two partitions:

```
[root@rhel-9 ~ #] fdisk /dev/sdb
Welcome to fdisk (util-linux 2.37.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (1-4, default 1):
First sector (2048-8388607, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-8388607, default 8388607): +1.5G

Created a new partition 1 of type 'Linux' and of size 1.5 GiB.

Command (m for help): n
Partition type
   p   primary (1 primary, 0 extended, 3 free)
   e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (2-4, default 2):
First sector (3147776-8388607, default 3147776):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (3147776-8388607, default 8388607):

Created a new partition 2 of type 'Linux' and of size 2.5 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

The size of the first partition is 1.5G. The size of the second partition is 2.5G.

Let's check it with `lsblk` command:

```
[root@rhel-9 ~ #] lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                 8:0      0   5G  0 disk
├─sda1                             8:1      0   5G  0 part
│   └─vg_test-lvtest1             253:1    0    2G  0 lvm  /dir1
│       └─vg_test-lvtest2         253:2    0    3G  0 lvm  /dir2
sdb                                 8:16     0   4G  0 disk
├─sdb1                             8:17     0  1.5G  0 part
└─sdb2                             8:18     0  2.5G  0 part
```

We can see our new partitions from the second disk.

12) BTW, the size of the first partition is 1.5G.

We need to use `vgextend <VG\_NAME> <PARTITION\_NAME>` to extend our existing VG:

```
[root@rhel-9 ~ #] vgs
VG      #PV #LV #SN Attr   VSize  VFree
rhel    1   1   0 wz--n-  8.41g    0
vg_test 1   2   0 wz--n- <5.00g    0
[root@rhel-9 ~ #] vgextend vg_test /dev/sdb1
Physical volume "/dev/sdb1" successfully created.
Volume group "vg_test" successfully extended
[root@rhel-9 ~ #] vgs
VG      #PV #LV #SN Attr   VSize  VFree
rhel    1   1   0 wz--n-  8.41g    0
vg_test 2   2   0 wz--n-  6.49g <1.50g
```

As you can see, the size of `vg\_test` has increased.

13) We need to use `-s` option to set physical extend. Full command: `vgcreate -s 16 vg_test2 /dev/sdb2`:

```
[root@rhel-9 ~ #] vgcreate -s 16 vg_test2 /dev/sdb2
Physical volume "/dev/sdb2" successfully created.
Volume group "vg_test2" successfully created
[root@rhel-9 ~ #] vgs
VG      #PV #LV #SN Attr   VSize  VFree
rhel    1   1   0 wz--n-  8.41g    0
vg_test 2   2   0 wz--n-  6.49g <1.50g
vg_test2 1   0   0 wz--n-  2.48g  2.48g
[root@rhel-9 ~ #] pvs
PV      VG      Fmt  Attr  PSize  PFree
/dev/nvme0n1p3 rhel    lvm2 a--   8.41g    0
/dev/sda1      vg_test lvm2 a--   <5.00g    0
/dev/sdb1      vg_test lvm2 a--   <1.50g <1.50g
/dev/sdb2      vg_test2 lvm2 a--   2.48g  2.48g
```

We can check with `vgs` whether the VG is created successfully or not. `pvs` command also shows that PV is created automatically when we run `vgcreate`.

If we want to see Physical Extend in our VG, we need to run `vgdisplay` command:

If we mount a disk to a directory permanently, we need to add a line about it to `/etc/fstab` file and run some specific commands. I have talked about it above. However, let's do it again and use `blkid`, `cut` commands and redirections (it is asked in the next task).

First, let's create a directory `/oradata` as specified in the task:

```
[root@rhel-9 ~ #] mkdir /oradata
```

Then we can use that command to add a new line to `/etc/fstab`:

```
[root@rhel-9 ~ #] echo "UUID=$(blkid /dev/vg_test2/lvtest2 | cut -d'"'"' -f2) /oradata xfs defaults 0 0" >> /etc/fstab
[root@rhel-9 ~ #] systemctl daemon-reload && mount -a
```

If I print the last line of `/etc/fstab`:

```
[root@rhel-9 ~ #] tail -n1 /etc/fstab
UUID=fad1d534-f306-49a7-9596-ae905e2433df /oradata xfs defaults 0 0
```

`cut` command is used to extract specific parts of each line from a file or input.

`blkid` command helps us to get information about UUID of the partition or logical volume and filesystem.

As a redirection, I used `>>`, not `>`. Because I do not need to overwrite. I want to append.

We need to reboot, or write `systemctl daemon-reload && mount -a` command to make changes applied.

- 14) I have used `blkid`, `cut` commands and redirection to add a line to `/etc/fstab` file in the previous task. If we add a line to `/etc/fstab` about a disk and mountpoint successfully, and reboot the system, that means the disks are successfully mounted. We can check it:

```
[root@rhel-9 ~ #] lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   5G  0 disk
├─sda1                              8:1      0   5G  0 part
│   ├─vg_test-lvtest1              253:1    0    2G  0 lvm  /dir1
│   └─vg_test-lvtest2              253:2    0    3G  0 lvm  /dir2
sdb                                  8:16     0   4G  0 disk
├─sdb1                              8:17     0  1.5G  0 part
└─sdb2                              8:18     0  2.5G  0 part
    └─vg_test2-lvtest2             253:3    0  2.5G  0 lvm  /oradata
```

As you can see, these three directories are mountpoints for my disks (LVs).

Rebooting is the best way to check whether the disks or logical volumes are successfully mounted or not. If there is no problem in the booting, that refers we have done it right!

- 15) As I mentioned above, we need to add the specific line to `/etc/fstab` file to mount disks permanently.

The syntax of fstab is like that:

Partition or logical volume path OR their UUID + mountpoint (directory) +
Filesystem + mount options (I prefer defaults) + value to allow or deny backup +
value to allow or deny filesystem check in the boot.

Task 2

- 1) Currently, there is no swap memory in my VM. So it is the first swap memory I will add.

Let's create VG from existing partition:

```
[root@rhel-9 ~ #] vgcreate vg01 /dev/sda1 #Created PV & VG
Physical volume "/dev/sda1" successfully created.
Volume group "vg01" successfully created
```

Let's create LV (I gave full VG space, because there is no limitation in the task):

```
[root@rhel-9 ~ #] lvcreate -n lv01 -l 100%FREE vg01
WARNING: ext4 signature detected on /dev/vg01/lv01 at offset 1080. Wipe it? [y/n]: y
Wiping ext4 signature on /dev/vg01/lv01.
Logical volume "lv01" created.
[root@rhel-9 ~ #] lvs
LV VG Attr LSize Pool Origin Data% Meta% Move Log Cpy%Sync Convert
root rhel -wi-ao---- 8.41g
lv01 vg01 -wi-a----- <5.00g
```

Then we need to use `mkswap` command to write its filesystem:

```
[root@rhel-9 ~ #] mkswap /dev/vg01/lv01
Setting up swapspace version 1, size = 5 GiB (5364510720 bytes)
no label, UUID=e8ce2e30-0c29-45bd-802f-34ef9b0194fe
```

- 2) First, let's take UUID for the LV:

```
[root@rhel-9 ~ #] blkid /dev/vg01/lv01
/dev/vg01/lv01: UUID="e8ce2e30-0c29-45bd-802f-34ef9b0194fe" TYPE="swap"
```

Then add this line to `/etc/fstab`:

```
UUID=e8ce2e30-0c29-45bd-802f-34ef9b0194fe none swap defaults 0 0
```

If we want to check whether the swap memory works successfully or not, we use

`free -mh` command:

```
[root@rhel-9 ~ #] free -mh
total used free shared buff/cache available
Mem: 1.6Gi 450Mi 1.1Gi 6.0Mi 275Mi 1.2Gi
Swap: 0B 0B 0B
```

As you can see, there is no swap memory yet. We will solve it in the next tasks.

- 3) As I mentioned in the previous task, we need to use `free -mh` command to display both RAM and SWAP memories.

If we want to 'activate' the swap memory, we need to either reboot the system or run these commands:

```
[root@rhel-9 ~ #] systemctl daemon-reload && swapon -a
[root@rhel-9 ~ #] free -mh #It will work now
total used free shared buff/cache available
Mem: 1.6Gi 454Mi 1.1Gi 6.0Mi 275Mi 1.2Gi
Swap: 5.0Gi 0B 5.0Gi
```

4) /etc/fstab is a **static and basic config file** where we write manually. It is basically a file to tell the system about permanent mounts. The mounts will work even after the reboot process.

/etc/mtab is a **dynamic file** that our system handles automatically. We never touch it. It is automatically updated whenever we mount or unmount something. On RHEL 9, /etc/mtab is soft link for to /proc/mounts.

5) It is mounting options which we can use. We can write ``ro`` which means read only. ``noauto`` is used skip boot mounting. ``_netdev`` means not to mount if there is no network.

6) As I mentioned above, it is ``_netdev``.

7) I can change it to this:

```
UUID=79f1223d-f24b-48f4-a869-b2352eabd13e /dir1 xfs ro,noexec 0 0
```

Then reboot the system to check whether everything is ok or not.

When I try to create any file inside ``dir`` directory, it is impossible:

```
[root@rhel-9 /dir1 #] touch f.txt  
touch: cannot touch 'f.txt': Read-only file system
```

If there is any executable file beforehand, we cannot execute it, because there is ``noexec`` too.

8) ``noexec`` does not allow any **binaries or scripts from being executed** from that mounted filesystem.

9) A partition table is a data structure stored on the disk which shows how our disk is divided and stores some data like partition start and end point in the memory. It is in the beginning of the disk where is called ``First Sector``. In MBR, we can create 4 primary partitions. OR we can create 3 primary and one extended partition which includes theoretically unlimited logical partitions.

10) There are two main limits which are solved in GPT. The first is disk size, so MBR lets us divide the disks with max size 2 TB. In GPT, it is up to 8 ZB. The second is partition count. MBR lets max 4 primary or 3 primary one extended partition. GPT it is 128.

11) When we run ``df -h``:

```
[root@rhel-9 /dir1 #] df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0    4.0M   0% /dev
tmpfs           838M   0    838M   0% /dev/shm
tmpfs           335M  6.4M   329M   2% /run
efivarfs        256K   57K   195K  23% /sys/firmware/efi/efivars
/dev/mapper/rhel-root 8.4G  2.4G   6.1G  29% /
/dev/nvme0n1p2   960M  244M   717M  26% /boot
/dev/nvme0n1p1   599M  7.4M   592M   2% /boot/efi
/dev/sr0         13G    13G     0 100% /mnt
tmpfs           168M  4.0K   168M   1% /run/user/0
/dev/sdb1        1.5G  256K   1.5G   1% /dir1
```

`/boot` cannot be on LV, because GRUB (which is bootloader) runs before LVM is initialized. Thus, GRUB does not have any info about LVM. The others can use LV, because they already know about LVM while booting.

Task 3

- 1) To create a new mountpoint:

ADD A NEW MOUNT POINT

More customization options are available after creating the mount point below.

Mount Point:

/boot

▼

Desired Capacity:

1G

Cancel

Add mount point

To create a new vg (vgproject):

CONFIGURE VOLUME GROUP

Please create a name for this volume group and select at least one disk below.

Name:

Description
VMware Virtual NVMe Disk (VMware_Virtual_NVMe_Disk_VMware_NVME_000)

☐ Encrypt

RAID Level:

Size policy:

MANUAL PARTITIONING

RED HAT ENTERPRISE LINUX 10.1 INSTALLATION

Done

us

DATA

/home

vgproject-lvhome

3 GiB

/opt

vgproject-lvopt

3 GiB

/u01

vgproject-lvu01

2 GiB

/var/log

vgproject-lvarlog

2 GiB

SYSTEM

/boot/efi

nvme0n1p1

600 MiB

>

/boot

nvme0n1p2

1024 MiB

/

vgproject-lvroot

3 GiB

/tmp

vgproject-lvtmp

2 GiB

/usr

vgproject-lvusr

5 GiB

/var

vgproject-lvar

2 GiB

swap

vgproject-lvswap

2 GiB

+ - ↺

AVAILABLE SPACE

9.41 GiB

TOTAL SPACE

35 GiB

1 storage device selected

nvme0n1p1

Mount Point:

/boot/efi

Device(s):

VMware Virtual NVMe Disk (nvme0n1)

Modify...

Desired Capacity:

600 MiB

Device Type:

Standard Partition

☐ Encrypt

File System:

EFI System Partition

☒ Reformat

Label:

Name:

nvme0n1p1

Update Settings

Note: The settings you make on this screen will not be applied until you click on the main menu's 'Begin Installation' button.

Discard All Changes

As you see, I give 5GB to /usr.

```

[root@localhost ~]# df -h
Filesystem                Size      Used Avail Use% Mounted on
devtmpfs                   4.0M         0   4.0M   0% /dev
tmpfs                      838M         0   838M   0% /dev/shm
tmpfs                      335M    5.1M   330M   2% /run
efivarfs                   256K     56K   196K  23% /sys/firmware/efi/efivars
/dev/mapper/vgproject-lvroot 3.0G     78M   2.9G   3% /
/dev/mapper/vgproject-lvusr  5.0G    1.6G   3.5G  31% /usr
/dev/nvme0n1p2             960M    244M   717M  26% /boot
/dev/nvme0n1p1             599M     7.4M   592M   2% /boot/efi
/dev/mapper/vgproject-lvopt  3.0G     54M   2.9G   2% /opt
/dev/mapper/vgproject-lvu01  2.0G     47M   1.9G   3% /u01
/dev/mapper/vgproject-lvvar  2.0G    111M   1.9G   6% /var
/dev/mapper/vgproject-lvtmp  2.0G     47M   1.9G   3% /tmp
/dev/mapper/vgproject-lvhome 3.0G     54M   2.9G   2% /home
/dev/mapper/vgproject-lvarlog 2.0G     58M   1.9G   3% /var/log
tmpfs                      168M         0   168M   0% /run/user/0

[root@localhost ~]# vgs
VG                #PV #LV #SN Attr   VSize  VFree
vgproject         1   9   0 wz--n- 33.41g 9.41g

[root@localhost ~]# lvs
LV                VG                Attr      LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
lvhome            vgproject         -wi-ao---- 3.00g
lvopt             vgproject         -wi-ao---- 3.00g
lvroot            vgproject         -wi-ao---- 3.00g
lvswap            vgproject         -wi-ao---- 2.00g
lvtmp             vgproject         -wi-ao---- 2.00g
lvu01             vgproject         -wi-ao---- 2.00g
lvusr             vgproject         -wi-ao---- 5.00g
lvvar             vgproject         -wi-ao---- 2.00g
lvvarlog          vgproject         -wi-ao---- 2.00g

```

To verify swap is active run:

```

[root@localhost ~]# swapon --show
NAME        TYPE      SIZE USED PRIO
/dev/dm-1   partition 2G      0B    -2

```